Name: ________ Cohort/Batch: ________ Date: ________ Score: ________

# SQLITE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Databases

TOTAL: 10 QUESTIONS

**Q1** What type of database is SQLite and what are its core strengths? *Threat Model*

**Q2** How does SQLite guarantee consistency and handle transactions? *Architecture*

**Q3** What index types does SQLite support and when do you use each? *Configuration*

**Q4** How do you design a schema or data model in SQLite? *Hardening*

**Q5** How does SQLite scale horizontally? *SSO / IdP*

**Q6** How do you monitor and tune slow queries in SQLite? *Observability*

**Q7** What backup and restore strategies does SQLite support? *Lifecycle*

**Q8** How do you handle schema migrations in SQLite? *Compliance*

**Q9** What security features does SQLite provide? *Testing*

**Q10** What are common anti-patterns when using SQLite in production? *Tuning*

## ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

### A1 SQLite is a relational, document, key-value,

Mitigation

SQLite is a relational, document, key-value, graph, or time-series store. Its strengths such as ACID guarantees, horizontal scale, or flexible schema guide the workloads it is best suited for.

### A2 SQLite provides either full ACID transactions

Primitives

SQLite provides either full ACID transactions or eventual consistency depending on its CAP theorem positioning. Transaction support varies from none (simple K/V stores) to serializable isolation (relational DBs).

### A3 SQLite offers B-tree, hash, full-text, spatial,

Hardening

SQLite offers B-tree, hash, full-text, spatial, or composite indexes depending on the engine. Choose based on query patterns: B-tree for range queries, hash for equality lookups, full-text for search.

### A4 Schema design in SQLite involves normalizing

Gotchas

Schema design in SQLite involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Understanding query needs upfront prevents costly migrations later.

### A5 SQLite scales via replication (read replicas)

Federation

SQLite scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Some SQLite implementations handle this automatically; others require manual configuration.

### A6 SQLite provides a slow query log,

Observability

SQLite provides a slow query log, explain/explain plan, or profiling tools to surface inefficient queries. Common fixes include adding indexes, rewriting queries, or increasing cache size.

### A7 SQLite supports logical backups (export SQL

Automation

SQLite supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Point-in-time recovery requires WAL/oplog archiving on top of backups.

### A8 Schema migrations in SQLite are managed

Governance

Schema migrations in SQLite are managed with versioned migration files applied in order by a migration tool. Migrations should be idempotent and tested in staging before production to avoid downtime.

### A9 SQLite supports role-based access control,

Assessment

SQLite supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Always restrict user privileges to the minimum needed and disable anonymous access in production.

### A10 Common mistakes include selecting all

Optimization

Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage.